Deploying EcoHome on Render

Every step needed to get the app, its database connection and Singpass/Corppass login (via MockPass) running on Render.

Overview

The deployment is two Render **Web Services** built from the same GitHub repo:

Service	What it runs	Address
Main app	The Express server, which also serves the built Vue client. The site and the API share one address, so the login cookie works.	https://<app>.onrender.com
MockPass	The Singpass/Corppass simulator you already use locally.	https://<mockpass>.onrender.com

Throughout this guide, replace **<app>** and **<mockpass>** with your two service names. Each service needs the other's address, so pick both names before you start.

> **Check the real addresses.** If a name is already taken, Render adds a suffix (for example *ecohome-a1b2.onrender.com*). After you create each service, copy the address shown at the top of its page and use that everywhere below.

What you need before starting

- A Render account connected to the team's GitHub repo.
- Access to the MongoDB Atlas project (to change Network Access).
- Your local `server/.env` file (for the values to copy).
- Your local `server/keys/singpass-jwks.json` file. If it doesn't exist, run `pnpm keys` inside `server/` to create it.

Step 1. Code change and push

Two problems in `server/server.js` stop the app working on Render, and both are fixed by the block below, which goes just before the final 404 handler at the end of the file:

- The server only listened on `127.0.0.1`, which Render can't reach (the deploy fails with *no open ports detected*). It now listens on `0.0.0.0` in production and stays on `127.0.0.1` locally.
- Express didn't serve the Vue app. It now serves `client/dist`, and sends `index.html` for any other page outside `/api` so Vue Router can handle it when someone refreshes the page.

```
// Serve the built Vue app (client/dist) so the site and /api share one origin.
// Any other GET outside /api gets index.html so Vue Router can handle it.
const CLIENT_DIST = path.join(__dirname, '..', 'client', 'dist');
if (fs.existsSync(CLIENT_DIST)) {
    app.use(express.static(CLIENT_DIST));
    app.use((req, res, next) => {
        if (req.method !== 'GET' || req.path.startsWith('/api')) return next();
        res.sendFile(path.join(CLIENT_DIST, 'index.html'));
    });
}
```

```
app.use((req, res) => {
    res.status(404).type('text/plain').send('Not found');
});

// Render needs the server reachable from outside the container;
// locally it stays on 127.0.0.1.
const HOST = isProduction ? '0.0.0.0' : '127.0.0.1';

// Start listening once MongoDB is connected.
dbReady
    .then(() => {
        app.listen(PORT, HOST, () => {
            console.log(`Server running at http://${HOST}:${PORT}/`);
        });
```

Then get the change onto the branch Render will deploy:

1. Commit the change to `server/server.js` and push it to GitHub.
2. Merge it into the branch you'll deploy from (usually `main`) through a pull request.

> Render only sees what is on GitHub. Uncommitted changes on your laptop never reach it. Never commit **.env** or **server/keys/**: both are gitignored on purpose.

Step 2. Let Render connect to MongoDB Atlas

Atlas refuses connections from addresses it doesn't know, and Render's free services don't have a fixed address.

1. In Atlas, open your project and go to **Security** → **Network Access**.
2. Click **Add IP Address**, then **Allow Access from Anywhere** (`0.0.0.0/0`), and confirm.

> The database is still protected by the username and password in the connection string. Keep that string secret.

Step 3. Create the main app service

In Render, click **New** → **Web Service**, pick the repo, and fill in:

Setting	Value
Name	<app> (for example *ecohome*)
Language	Node
Branch	the branch from Step 1 (usually `main`)
Root Directory	leave blank (the repo root)
Build Command	`cd client && pnpm install --prod=false && pnpm build && cd ../server && pnpm install`
Start Command	`cd server && node server.js`
Instance Type	Free is fine

> **Why `--prod=false`:** once `NODE_ENV=production` is set (Step 4), pnpm skips devDependencies, and Vite is one of them. Without this flag the client build fails with *vite: not found*.

Step 4. Main app environment variables

On the main app service, open **Environment** and add each of these. Values marked *from .env* are copied from your local `server/.env`.

Key	Value
`NODE_ENV`	`production`. Turns on secure cookies and proxy support; required for logins to stay signed in over HTTPS.
`NODE_VERSION`	`24`. The client needs Node 22.18 or newer.
`DB`	from .env (your Atlas connection string)
`SESSION_SECRET`	a **new** random string, generated with the command below
`GEMINI_API_KEY`	from .env
`SINGPASS_CLIENT_ID`	from .env (any value works with MockPass)
`SINGPASS_SCOPES`	from .env
`SINGPASS_DISCOVERY_URL`	`https://<mockpass>.onrender.com/singpass/v3/fapi/.well-known/openid-configuration`
`SINGPASS_REDIRECT_URI`	`https://<app>.onrender.com/api/auth/singpass/callback`
`CORPPASS_DISCOVERY_URL`	`https://<mockpass>.onrender.com/corppass/v2/.well-known/openid-configuration`
`CORPPASS_REDIRECT_URI`	`https://<app>.onrender.com/api/auth/corppass/callback`
`SINGPASS_KEYS_FILE`	`/etc/secrets/singpass-jwks.json` (see Step 5)

Generate the session secret on your laptop and paste the output into Render:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

> **Don't add `PORT`.** Render sets it itself. The Corppass variables aren't in your local .env because the code defaults to localhost; on Render they must be set.

Step 5. Upload the login keys as a Secret File

`server/keys/singpass-jwks.json` holds the app's private keys for Singpass/Corppass. It is gitignored, so Render doesn't have it.

1. On the main app service, open **Environment** → **Secret Files** → **Add Secret File**.
2. Filename: `singpass-jwks.json`
3. Contents: open `server/keys/singpass-jwks.json` in VS Code, select all, and paste.
4. Save. Render makes the file available at `/etc/secrets/singpass-jwks.json`, which is what `SINGPASS_KEYS_FILE` points to.

Step 6. Create the MockPass service

Click **New** → **Web Service** again and pick the **same repo**:

Setting	Value
Name	<mockpass> (for example *ecohome-mockpass*)
Language	Node
Root Directory	`server`
Build Command	`echo "no build"`
Start Command	`npx -y @opengovsg/mockpass@4.9.3`
Instance Type	Free is fine

Then add these environment variables to the MockPass service. They match your local `pnpm mockpass` script, with the main app's Render address in place of `127.0.0.1:8000`:

Key	Value
`SHOW_LOGIN_PAGE`	`true`
`FAPI_CLIENT_JWKS_ENDPOINT`	`https://<app>.onrender.com/api/auth/singpass/jwks`
`CP_RP_JWKS_ENDPOINT`	`https://<app>.onrender.com/api/auth/singpass/jwks`

> Both point at the **singpass** JWKS address. That is intentional: Corppass uses the same keys, exactly as in your local script.

Step 7. Deploy and check

If you changed any addresses after the first deploy, use **Manual Deploy** → **Deploy latest commit** on both services so they pick up the new values. Then check each of these in your browser:

Open	You should see
`https://<app>.onrender.com`	The EcoHome site.
`https://<app>.onrender.com/api/auth/singpass/jwks`	JSON with two public keys. An error means Step 5 isn't set up correctly.
`https://<mockpass>.onrender.com/singpass/v3/fapi/.well-known/openid-configuration`	JSON in which every address starts with `https://<mockpass>.onrender.com`.
Log in with Singpass on the site	The MockPass persona page, then back to the site, signed in.
Log in with Corppass on the site	The same, for a contractor account.

Troubleshooting

Read the service's **Logs** tab first. These are the likely messages:

Symptom	Fix
Deploy fails: *No open ports detected*	The Step 1 change isn't on the deployed branch, or `NODE_ENV` isn't `production`.
Build fails: *vite: not found*	The build command is missing `--prod=false` (Step 3).
Build fails: *pnpm: command not found*	Replace each `pnpm` in the build command with `npx -y pnpm@10`.
Build fails with a Node engine error	Set `NODE_VERSION` to `24` (Step 4).
Crash: *SESSION_SECRET must be set in production*	Add `SESSION_SECRET` (Step 4).
Crash: *MongoDB error: ...*	Check `DB` is copied exactly, and that Atlas allows `0.0.0.0/0` (Step 2).
Login error: *Login keys not found at ...*	Add the secret file and `SINGPASS_KEYS_FILE` (Step 5).
The first Singpass/Corppass login fails, the next one works	MockPass was asleep. Free services sleep after about 15 minutes idle and take up to a minute to wake. Open the MockPass address before a demo.
Signed in, but immediately signed out again	Check `NODE_ENV` is `production` and you're using the `https://` address.
Refreshing a page like /profile shows *Not found*	The client build didn't run, so `client/dist` is missing. Check the build log.